

Requirement Understanding Document

User Posts and Albums Viewer

RUD v1.1 | Requirements confirmed and updated

Field	Details
Document Version	1.1
Status	Requirements confirmed; UI clarification incorporated
Application Type	Client-side / presentation-layer web application
Technology	HTML5, CSS3, JavaScript (ES6+), jQuery 4.0.0
Data Source	JSONPlaceholder REST API
Prepared For	Requirement confirmation
Prepared By	Atul Sharma
Date	03 October 2026

Purpose of this document

This revision incorporates the confirmed requirements: display the user's first name and last name, and present Posts and Albums separately. A high-level UI wireframe has also been added to clarify the intended feature structure.

1. Project Overview

The assignment is understood as a small client-side web application that consumes the JSONPlaceholder REST API. The application will retrieve a list of users, present those users through a web interface, and when a user is selected, retrieve and display that user's posts and albums. The application must also provide loading and error handling feedback.

2. Objective

- Retrieve user records from JSONPlaceholder and present them in a usable interface.
- Allow the user to select a specific user from the displayed list.
- Retrieve the selected user's posts and albums using the user ID.
- Present the retrieved posts and albums without requiring a full page reload for each selection.
- Provide clear loading feedback while requests are in progress and appropriate error feedback when requests fail.

3. Scope

3.1 In Scope

- User retrieval from the JSONPlaceholder API.
- Display each user using first name and last name.
- Dynamic display of users in the browser.
- User click/selection interaction.
- Retrieval of posts and albums associated with the selected user.
- Present Posts and Albums separately through the UI.
- Dynamic display of posts and albums.
- Loading-state handling.

- Update the client-side UI dynamically without requiring a full page reload for each user selection.
- Handle API/request errors and provide an appropriate user-facing message.
- Basic responsive and presentable styling; no strict visual design has been specified.

3.2 Out of Scope

- Custom backend development.
- Custom database or persistent storage.
- Authentication and authorization.
- CRUD operations against a custom data store.
- Administrative features.
- Authentication, authorization, registration, or login.
- Advanced UI/UX design beyond a clean, functional presentation layer.

4. Technology and Architecture Understanding

The assignment is understood to be a client-side/presentation-layer website using HTML and jQuery, with CSS for basic presentation. jQuery will handle interaction, AJAX/API communication, response processing, and DOM updates. JSONPlaceholder is the external data source; no custom backend or database is currently in scope.

Layer / Component	Responsibility
Presentation layer	HTML/CSS structure and visual presentation of users, posts, albums, loading states, and errors.
Client-side application logic	jQuery event handling, API requests, response processing, validation of HTTP success, and DOM updates.
External API	JSONPlaceholder provides user, post, and album data through REST-style HTTP endpoints.

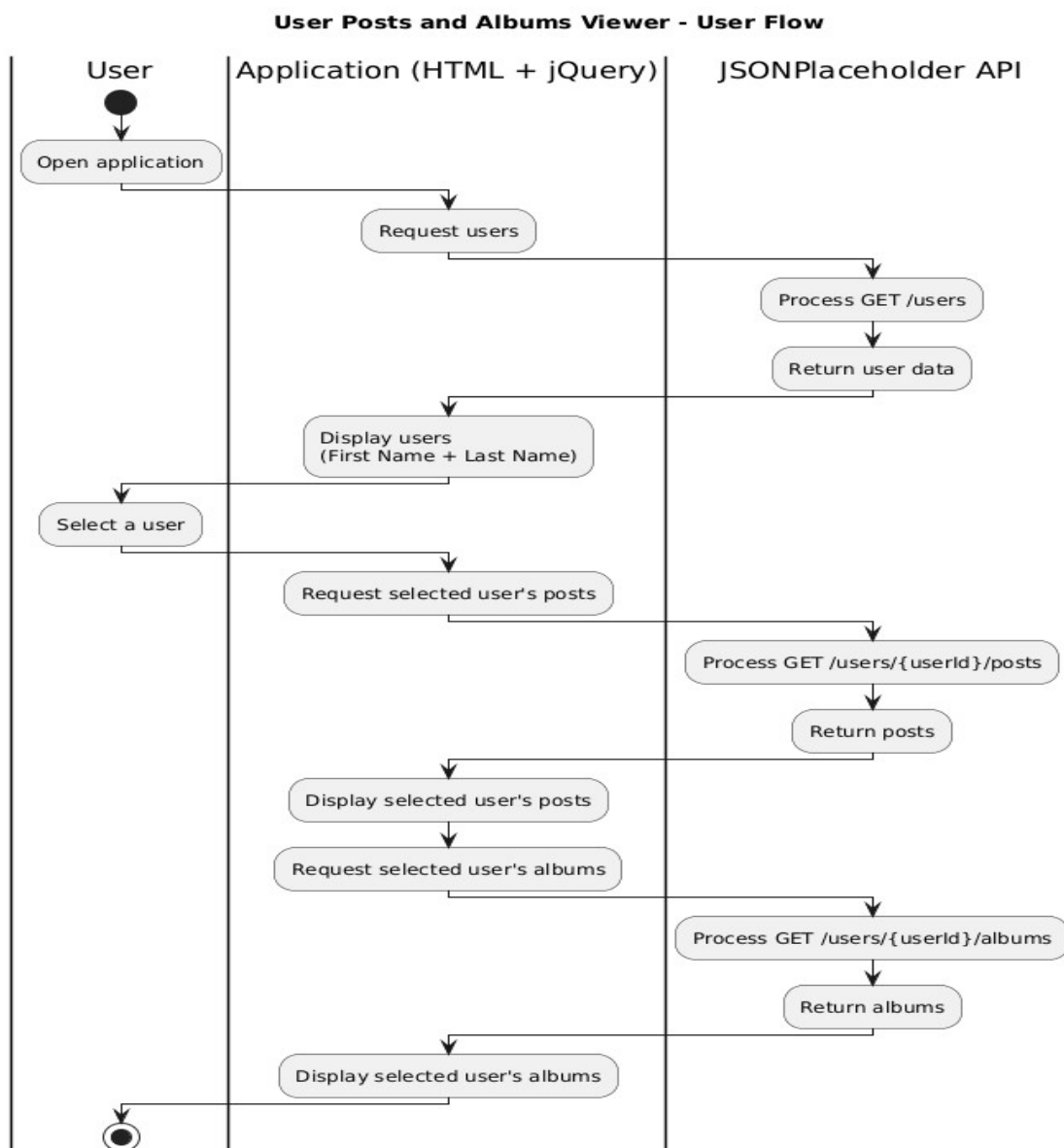
5. Functional Requirements

ID	Requirement	Description	Status
FR-01	Retrieve users	Retrieve users from the JSONPlaceholder /users endpoint.	Confirmed
FR-02	Display users	The application shall display the first name and last name of each retrieved user.	Confirmed
FR-03	Select user	Allow the user to select/click a displayed user.	Confirmed
FR-04	Retrieve posts	After selection, retrieve posts associated with the selected user ID.	Confirmed
FR-05	Retrieve albums	After selection, retrieve albums associated with the selected user ID.	Confirmed
FR-06	Display related data	Display the selected user's posts and albums. Posts and Albums shall be presented separately; they shall not be combined into one list/view.	Confirmed
FR-07	Loading state	Show an appropriate loading indication while API data is being retrieved.	Confirmed

ID	Requirement	Description	Status
FR-08	Error handling	Handle API/request failures with an appropriate user-facing error message.	Confirmed
FR-09	Dynamic update	Selecting another user updates the related data without a manual full-page refresh.	Confirmed

6. User Flow

The following requirement flow captures the current understanding of the requested behavior. Detailed technical architecture will be documented later in the HLD/LLD.



7. API Understanding

The following API resources are currently expected to be used:

<https://jsonplaceholder.typicode.com/users> - Retrieve users

<https://jsonplaceholder.typicode.com/users/{id}/posts> - Retrieve posts for the selected user

<https://jsonplaceholder.typicode.com/users/{id}/albums> - Retrieve albums for the selected user

8. Non-Functional Requirements

ID	Requirement	Expected Outcome
NFR-01	Usability	The interface should make user selection and viewing of related data straightforward.
NFR-02	Responsiveness of feedback	The UI should indicate when data is being loaded and avoid leaving the user uncertain about request state.
NFR-03	Error handling	Failures should be handled gracefully with meaningful feedback rather than leaving stale or broken UI.
NFR-04	Maintainability	Client-side code should separate API communication, user interaction, and rendering responsibilities where practical.
NFR-05	Browser compatibility	The application should work in commonly used modern browsers.

9. Confirmed Assumptions and Constraints

9.1 Confirmed Assumptions

- JSONPlaceholder is the only data source required for the assignment.
- The application is client-side/presentation focused and does not require a custom backend or database.
- The user list will display first name and last name only.
- Posts and Albums will be presented separately after user selection.
- The visual design is flexible because no strict styling requirement was specified.

9.2 Constraints

- HTML and jQuery are explicitly required technologies for the assignment.
- The application must rely on JSONPlaceholder for user, post, and album data.
- Loading and error handling are required.

10. High-Level User Interface

A high-level wireframe is included below to clarify the major UI areas. The wireframe intentionally avoids detailed visual styling and focuses on feature structure and interaction.

USER POSTS & ALBUMS VIEWER

Users

Leanne Graham

Ervin Howell

Clementine Bauch

Patricia Lebsack

Selected User: Leanne Graham

Posts

Albums

Posts and Albums are displayed separately in the same content area based on the selected option.

Albums

quidem molestiae enim

sunt qui excepturi placeat culpa

omnis laborum odio

non esse culpa molestiae omnis sed optio

Note: Example data shown for UI representation.

Loading state: "Loading posts..." or "Loading albums..." appears while data is being fetched.

Error state: An error message appears if the API request fails.

11. Status and Next Action

Current Status

Requirements are confirmed based on clarification. The RUD has been updated to reflect first-name/last-name display, separate Posts and Albums presentation, and the requested high-level UI wireframe.

User Posts and Albums Viewer • Page 5